
VIM
MOTIONS
Vim motions

Introduction

Who even is this guy?

- Senior Software Engineer @iHeart
- Keyboard and ergonomics enthusiast
- Die hard terminal fanboy

What is this vim thing?

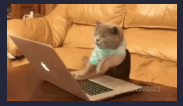
Well it is not a gym...

"Vim is a powerful, free, and open-source terminal based text editor known for its efficiency and keyboard-centric approach." - ChatGPT

Why am I here?

What is this talk *not* about?

- Setting up the vim editor in your terminal
- An in-depth guide into everything you can do with vim
- Abolishing mice and touch devices
  - Although this one is borderline



What is this talk about then!?

- Using the modal style editing that vim embraces to make moving around text documents and projects *blazingly* **fast**
- Getting a basic understanding of the core building blocks of cursor movement in vim, aka motions
- Talking about where you can use vim motions outside of the terminal editor (Like plugins for other editors/IDEs)
- Most importantly, how to exit vim!

Why vim motions?

Why use vim motions when I can just move my mouse?

Computer work and typing is strenuous activity!

It's estimated that 1.8 ***million*** US workers suffer from Repetitive Strain Injuries (RSI) every year.[1]

Researchers in Sweden has found that *half* of people who work with computers complain of RSI.[2]

Alright, but then what about this "blazing fast" text editing?

This all boils down to Actions Per Minute (APM). Knowing your way around the keyboard and all the keyboard shortcuts just lets you do things faster than clicking with a mouse.

So then I should just learn my editor's shortcuts?

I mean yes, you should. Especially if your editor has things like built in debugging or compilation features.

This talk is mostly focused on editing text though, so let's take a deeper dive into the differences between conventional text editors and modal text editors.

Let's talk modes

What is modality?

It is the state of being modal. Basically it just means to have different modes or patterns of interaction. Kind of like a transformer!

So what modes does vim have?

A whole bunch actually. However, the 4 most common are:

- `normal`
  - Where you can enter shortcuts for moving around documents, performing commands, etc. This is usually the default mode.
- `insert`
  - Where you can actually enter text. The most common keybind to enter `insert` mode is `i`
- `visual`
  - Allows you to visually select text using the motions similar to `normal` mode. The most common keybind to enter visual mode is with the `v` key, or `V` to enter `visual block` mode which lets you easily select entire lines.
- `command`
  - Gives you a sort of command palette to run vim commands. The most common keybind to enter `command` mode is `:`.
  - This is how you might normally *exit* vim, using the command `wq` to save (or *write*) and quit, or with `q!` to force quit.

I like to move it

So now that we have a small glimpe into the modes of vim, let's look more at vim motions and commands

A vim command is how to tell vim to take actions (such as move up one line, delete a word, etc). A vim motion is a subset of vim commands that allows you to move your cursor around a buffer (a.k.a. an open file). Some more common motions are:

Horizontal motions

- h/l
  - These move the cursor left and right respectively.
- 0
  - Brings you to the beginning of the line.
- ^
  - Brings you to the first non-whitespace character of the line.
- $
  - Brings you to the end of the line.
- f{char}
  - Brings you to the next occurrence of {char}.
- F{CHAR}
  - Brings you to the previous occurrence of {char}.
- t{char}
  - Brings you 1 char before the next occurrence of {char}.
- T{CHAR}
  - Brings you 1 char after the previous occurrence of {char}.

Vertical motions

- j/k
  - These move the cursor down and up respectively.
- G
  - Takes you to the last line of the buffer.
- gg
  - Takes you to the first line of the buffer.
- ctrl + f/b
  - Moves you down or up one full page
- ctrl + d/u
  - Moves you down or up one half page
- :{num}<CR>
  - Go to line number {num}.

I like to move it pt2

Text objects

- One of the really neat things about vim is the idea of text objects. These are things like words, sentances, paragraphs, and even coding specific objects like functions, classes, or blocks.

Word motions

These operate on individual words.

- w
  - Move forward one *word*.
- b
  - Move backward one *word*.
- e
  - Move forward to the end of the *word*.
- ge
  - Move backward to end of the previous *word*.

Sentance motions

- These operate on numbers of words until an ending punctuation (Such as ., !, or ?).
-)
  - Move forward one *sentance*.
- (
  - Move backwards one *sentance*.

Paragraph motions

- These operate on collections of sentances until an empty line.
- }
  - Move forward one *paragraph*.
- {
  - Move backwards one *paragraph*.

I like to SELECT it

Text object selections

These allow you to select a surrounding text object or block to perform an action on. These only work either in visual mode or after an operator (We'll get to those next)

- a
  - Selects an object.
- i
  - Selects inside an object.
- Examples
  - aw
    - Select a word
  - ip
    - Select inside a paragraph
  - i[
    - Select inside of a [] block

Now do it again

Vim motions can often be accompanied by a `count` that allows you to do the same motion multiple times without having to press the keybind more than once. This usually takes the form of `{count}{motion}`. Here are a couple examples:

- `3w`
  - Move forward 3 words.
- `5j`
  - Move down 5 lines.
- `2f-`
  - Move to the *2nd* occurrence of the `-` character.

Operator PLZ

The next classification of vim commands are operators

operators allow you to, well, *operate* on text. Things like, deleting, replacing, yanking (or copying), etc. These commands are combined with motions to allow you to do operations over a specific subset of text without having to select it first. These take the form of {operator}{motion}.

- c
  - "change"
  - Edits the text selected by the motion.
  - Example
    - cw
    - Changes the next word.
- d
  - "delete"
  - Deletes the text selected by the motion.
  - Example
    - d2w
    - Deletes the next 2 words.
    - There is a common shortcut of dd to delete an entire line.
- y
  - "yank"
  - Copies the text selected by the motion into the selected register (Default is ")
  - Example
    - yip
    - Yanks inside the current paragraph
    - There is a common shortcut of yy to yank an entire line.

On Your Mark

Marks let you save a spot in your file to easily jump back to

- To save a mark you use `m{char}` where char is some key like a, b, or 1.
- To recall a mark you have two options
  - `'{char}` goes to the first non-whitespace character on the line of the mark
  - `` `{char} `` goes to the exact cursor position of the mark
    - *I know, it won't let me highlight that backtick...*
- `:marks` to show a list of all your current marks

There are also some special marks

- `` `. `` jump to position where last change occurred in current buffer
- `` `" `` jump to position where last exited current buffer
- `` `0 `` jump to position in last file edited (when exited Vim)

You can use a mark as a motion too

- d`a
  - delete all text from current cursor potion to position of mark `a`
- d'a
  - delete all text from current line to line of mark `a`

And you can jump between them

-]' jump to next line with a mark
- [' jump to previous line with a mark
-]` jump to next mark
- [` jump to previous mark

Plus you can use `counts` to do them multiple times: `5]'` jumps the next 5 lines with marks

I Think I Need a Tutor

Learning this new paradigm of text editing can seem daunting. However there are 2 big advantages you have. They are vim tutor and emulation plugins for existing editors.

▒▒ Tutor me senpai

Vim comes with a really helpful sort of "tutorial" that goes over all this and more. You can invoke it from within the vim editor by running vimtutor from your shell. This will bring you into a tailored vim experience that will let you practice your skills and build muscle memory.

▒▒▒ But I can't be spending all my time in the tutorial, I need to actually get stuff done

I hear you on this one. This is where vim emulation plugins come in super handy! I started off using the VSCodeVim [3] plugin to learn these motions without leaving my existing setup. There are also plugins for JetBrains IDEs [4] and Sublime Text [5]. Zed even has native vim emulation [6]! Chances are there is an plugin for your editor of choice.

FIN

Citations

1. **ergonomics** (*https://www.shrm.org/resourcesandtools/hr-topics/risk-management/pages/ergonomics-away-from-office.aspx*)
2. **rsi** (*http://www.rsi.org.uk/pdf/ULDs_Overview.pdf*)
3. **VSCodeVim** (*https://marketplace.visualstudio.com/items?itemName=vscodevim.vim*)
4. **IdeaVim** (*https://lp.jetbrains.com/ideavim/*)
5. **vintage** (*https://www.sublimetext.com/docs/vintage.html*)
6. **Zed** (*https://zed.dev/docs/vim*)